1. You are given a directed graph $G = (V, E)$ with positive length edges, as well as two vertices s and t. An edge $e \in E$, is considered bad if the cost of all walks from s to t that uses e costs at least 3β, where β is the length of the shortest path from s to t. Describe an algorithm that computes all the *bad* edges in G. Slower algorithms would earn 60% of the total points.

Solution: Let's write some notations first.

- l_e is the length of edge $e \in E$. It is given that $l_e > 0$.
- The cost of a path from s to t through edges $e_1, e_2, \ldots e_n$ can be written as $\sum_{i=1}^{n} l_{e_i}$.
- Dijkstra(G, s, t) returns the shortest path between vertices s and t of a graph G. Assume that it is given to us, and it returns $-\infty$ if there is no path from s to t.

First, let's find β. We can use Dijkstra's algorithm to find the shortest path between s and t, and its cost would be β. We can use Dijkstra(G, s, t).

One way to solve the problem is to check shortest paths from s and t through all e's in E. If $e = (u, v)$, the shortest path from s to t through e can be computed as

$$\text{Dijkstra}(G, s, u) + l_e + \text{Dijkstra}(G, v, t)$$

All the edges through which the shortest path has a cost $> 3\beta$ would be the bad edges of G.

```
FindBadEdges(G, s, t):
  es ← φ
  β ← Dijkstra(G, s, t)
  for each e in G.Edges:
      α ← Dijkstra(G, s, e.u) + e.length + Dijkstra(G, e.v, t)
      if α > 3β:
          es.add(e)
  return es
```

The run time of this solution is $O(|E| \cdot (|E| + |V| \cdot \log |V|))$, assuming Dikstra's algorithm has a time complexity of $O(|E| + |V| \cdot \log |V|)$.

Can we improve the time complexity? Yes! Notice that while computing β, we can store the shortest path from s to $v \in V$ in a lookup table, which means Dijkstra$(G, s, e.u)$ may be replaced by a table lookup which can be done in $O(1)$. Creating a similar lookup table for Dijkstra$(G, e.v, t)$ is slightly tricky. We can create another graph $G' = (V, E')$ with $E' = \{(v, u) \mid (u, v) \in E\}$. We can run Dijkstra's algorithm as Dijkstra(G, t, s) and create a lookup table to store the length of the shortest paths from t to $v \in V$. This would reduce the time complexity of the solution to $O(|E| + |V| \cdot \log |V|)$ with an additional space complexity of $O(|V| + |E|)$. ■

2. You are given the following mystery recurrence. Your task is to determine what it computes. Once understood, provide a one-sentence English description of the recurrence SP and analyze its, memoized, most optimized runtime.

Recurrence Definition:

Base Case 1:

$$\text{SP}(u,v,k) = 0 + M(u,u,k) \quad \text{if } u = v \text{ and } k \geq 0$$

Base Case 2:

$$\text{SP}(u,v,k) = \infty + M(u,u,k) \quad \text{if } k < 0$$

Recursive Case:

$$\text{SP}(u,v,k) = \min\left(\text{SP}(u,v,k-1) + M(u,u,k), \min_{(x,v)\in E}\left[\text{SP}(u,x,k-1) + w(x,v) + M(u,u,k)\right]\right) + M(u,v,k)$$

$$M(u,v,k) = \begin{cases} M(u^2+1, v^3+6, k-1) & \text{if } k > 0 \\ 0 & \text{if } k \leq 0 \end{cases}$$

Solution: Step-by-step analysis:

- The function $M(u,v,k)$ looks recursive and nonlinear, but it only depends on k and always evaluates to 0.
- The function $\text{SP}(u,v,k)$ computes the shortest path from node u to node v using at most k edges, with extra M terms added that evaluate to zero.

One-sentence English description:

$\text{SP}(u,v,k)$ computes the length of the shortest path from u to v using at most k edges.

Runtime Analysis:

- Let V be the number of nodes, E the number of edges, and K the edge limit.
- The number of subproblems is $O(V^2 \cdot K)$.
- Each subproblem takes $O(\deg(v))$ time to compute due to the inner minimization over incoming edges.

Total Runtime: $\boxed{O(K \cdot E)}$

■